Autómatas, Teoría de Lenguajes y Compiladores

Proyecto Especial

Instituto Tecnológico de Buenos Aires



Integrantes:

- Ponce, Facundo (65288)
- Holgado, Matias (64727)
- Sanchez Novelli, Matias (65471)
- Rodriguez, Lucas (63335)

Fecha de Entrega: 09/11/2026

Tabla de Contenidos

1. Introducción....3
2. Modelo Computacional....3
2.1. Dominio....3
2.2. Lenguaje....4
3. Implementación....5
3.1. Frontend....5
3.2. Backend....6
3.3. Adicionales (opcional)....8
3.4. Dificultades Encontradas....8
4. Futuras Extensiones....8
5. Conclusiones....9
6. Bibliografía....9

1. Introducción

El presente trabajo consiste en el desarrollo de un compilador para TabularDSL, un lenguaje que permite describir y utilizar transformaciones de carga, filtrado, derivación de nuevas columnas, proyección, ordenamiento, truncado y guardado sobre archivos con formato .CSV (Comma Separated Values) para la materia de Autómatas, Teoría de lenguajes y Compiladores del Instituto tecnológico de Buenos Aires.

El compilador fue desarrollado en el lenguaje de programación C. Una de las principales herramientas utilizadas fue Flex para la definición y generación del analizador léxico, el cual se encarga de identificar los distintos componentes léxicos del lenguaje de entrada. Otra de las herramientas que se utilizaron fue Bison, que se utilizó para construir el analizador sintáctico para validar que la estructura del programa de entrada cumpla con la gramática definida, además de construir las estructuras necesarias para la posterior generación del programa de salida. Como lenguaje de salida se eligió Python, utilizando únicamente su biblioteca estándar.

El proceso se separó en tres etapas. La primera consistió en la propuesta y diseño de la idea principal, donde se pensaron requerimientos, ejemplos y casos de prueba. En la segunda etapa, se llevó a cabo el desarrollo del frontend del compilador, es decir el analizador léxico y sintáctico. El principal objetivo de esta etapa es la construcción del AST (Árbol de Sintaxis Abstracta). En la tercera etapa se desarrolló el backend del compilador, el cual consiste en realizar distintos tipos de validaciones al programa de entrada, para luego poder generar el código de salida.

2. Modelo Computacional

El proceso de desarrollo del compilador se basó en tres etapas, como se mencionó anteriormente. En el proceso de diseño, dos aspectos fundamentales que se determinaron fueron el dominio y el lenguaje.

2.1. Dominio

El dominio del compilador desarrollado en este proyecto se centró en la transformación de archivos de datos tabulares en formato .CSV. Actualmente este tipo de tareas de limpieza y extracción de datos se suele resolver con scripts escritos en lenguajes de propósito general o con cadenas de herramientas de línea de comandos. Este enfoque presenta varios problemas: cada persona reescribe la misma lógica, la transformación se mezcla con los detalles de lectura y escritura del archivo, y no existe una forma uniforme de compartir una transformación entre proyectos. Por este

motivo, se diseñó un lenguaje declarativo en el cual el usuario especifica qué transformación desea realizar, y el compilador se encarga de cómo ejecutarla.

Para acotar la complejidad del proyecto se tomaron las siguientes decisiones:

- El esquema del archivo de entrada (nombres de las columnas) no se declara en el programa, sino que se obtiene del encabezado del CSV.
- Los valores de las celdas se modelan con tres tipos: number, string y boolean.

2.2. Lenguaje

Un programa de TabularDSL se compone de una lista opcional de declaraciones (constantes, funciones y pipelines), seguida de un único script o de un único for. Todo script comienza obligatoriamente con una instrucción LOAD, seguida de una secuencia de stages que se aplican en orden sobre la tabla cargada, y termina con una instrucción SAVE. A cada operación sobre la tabla se le asignó una palabra reservada propia, como se muestra a continuación en un ejemplo que carga un archivo de usuarios, se queda con los mayores de edad activos y guarda únicamente sus correos electrónicos:

```
LOAD "users.csv"
FILTER @age >= 18 AND @status = "active"
SELECT email
SAVE "active_adults.csv"
```

Las palabras reservadas son case-insensitive, y las referencias a columnas dentro de expresiones se escriben con el prefijo @, para distinguirlas de los identificadores declarados en el programa. El programa anterior se traduce al siguiente código Python:

```
    rows = _load("users.csv")
    rows = [row for row in rows if ((row["age"] >= 18) and
(row["status"] == "active"))]
    rows = [{"email": row["email"]} for row in rows]
    _save(rows, "active_adults.csv")
```

Se puede apreciar que el programa de entrada es más corto y legible que su equivalente, y que el usuario no necesita conocer los detalles de lectura y escritura del archivo.

A nivel sintáctico, se implementaron las siguientes construcciones:

- Filtrado de filas (FILTER) mediante condiciones booleanas con operadores relacionales (=, !=, <, <=, >, >=), operadores lógicos (AND, OR, NOT) y paréntesis.
- Creación de nuevas columnas (MUTATE) a partir de expresiones aritméticas, incluyendo la forma condicional IF … THEN … ELSE, donde ambas ramas son obligatorias.
- Proyección de columnas (SELECT), ordenamiento (SORT BY … ASC/DESC) y truncado del resultado (LIMIT).
- Concatenación de strings mediante el operador +
- Constantes (let), funciones de una única expresión de retorno (function) y pipelines parametrizables (pipeline), que se invocan con APPLY.
- Restricciones sobre los parámetros de un pipeline (require), ubicadas en su cabecera.
- Iteración sobre una lista de variantes (for), incluyendo tuplas para ligar varios identificadores por iteración.
- Bloques de ramificación, que permiten generar varias salidas a partir de un único LOAD.
- Parámetros externos ($nombre), cuyo valor se provee al ejecutar el programa.
- Comentarios de línea con //.

3. Implementación

Para la implementación del compilador se utilizó como base el proyecto Flex-Bison-Compiler (sobre la rama development), que facilitó el uso de las herramientas Flex (para el análisis léxico) y Bison (para el análisis sintáctico), como ya se describió previamente.

3.1. Frontend

Para desarrollar el frontend, se definió una gramática formal que representara el lenguaje deseado. De esta manera se definieron como símbolos terminales: números, strings, booleanos, identificadores, referencias a columnas, parámetros externos y diversos tipos de tokens para las palabras reservadas, los operadores y los signos de puntuación. Para que las palabras reservadas fueran case-insensitive se utilizó la opción caseless de Flex. Además, los valores llegan limpios desde el analizador léxico: los strings sin comillas y las columnas y parámetros externos sin su prefijo.

A su vez se definieron como símbolos no terminales las siguientes estructuras:

- Las declaraciones de constantes, funciones y pipelines, junto con sus listas de parámetros y de argumentos.
- Las distintas stages (FILTER, MUTATE, SELECT, SORT, LIMIT, APPLY) y los bloques de ramificación.
- La instrucción for, junto con sus variantes y tuplas.
- Las expresiones aritméticas y las condiciones booleanas, separadas en varios niveles para reflejar la precedencia de los operadores directamente en la gramática.
- El símbolo inicial, definido como un program, compuesto por una lista de declaraciones seguida de un script o de un for.

Se intentó que distintos no terminales compartieran el mismo tipo de nodo en el AST siempre que fuera posible. Por ejemplo, las listas de parámetros, de columnas de un SELECT y de variables de un for se representan con una misma lista de identificadores, y las condiciones de FILTER y de require con un mismo tipo de nodo. Todas las listas del AST se modelaron como listas enlazadas, y los nodos con variantes como estructuras con un campo de tipo y una unión.

Desde el frontend fue posible validar muchas estructuras, como la obligatoriedad de LOAD al inicio y de SAVE al final de cada script o bloque, la exhaustividad del MUTATE condicional, o que la lista de variantes de un for no sea vacía. Sin embargo, hubo varias reglas que no dependen de la forma del programa sino del contexto, y que se debieron postergar para el análisis semántico del backend.

3.2. Backend

La primera etapa del desarrollo backend del compilador consistió del análisis del lenguaje elaborado previamente para poder determinar adecuadamente los elementos de la tabla de símbolos, donde se guardó información semántica relevante de los identificadores para realizar validaciones.

La tabla de símbolos se implementó como una pila de tablas de hash, donde cada tabla representa un scope. Se abre un scope nuevo para el cuerpo de cada función, de cada pipeline y de cada for, lo que permite que un parámetro oculte a una constante global del mismo nombre. Cada símbolo guarda su clase (constante, función, pipeline, parámetro o variable de un for) y su tipo. Para que las etapas posteriores no tuvieran que reconstruir los scopes una vez cerrados, se decidió que cada uso de un identificador en el AST guarde un puntero al símbolo al que se refiere.

Previo a la generación de código, se realizaron entonces las siguientes validaciones:

- Todo identificador utilizado debe estar declarado, y no puede redeclararse dentro de un mismo scope.
- Una constante no puede referenciarse a sí misma ni utilizar una constante declarada después.
- Las llamadas a funciones y las invocaciones con APPLY deben referirse a una función o pipeline existente, respectivamente, y respetar su aridad. Lo mismo aplica a las tuplas de un for respecto de la cantidad de variables.
- Las referencias a columnas solo se permiten dentro de FILTER y MUTATE, ya que son las únicas stages que se evalúan fila por fila.
- Las condiciones no pueden contener llamadas a funciones.
- Los argumentos de LOAD y SAVE deben ser de tipo string, y el de LIMIT de tipo number.
- No se permite aritmética sobre strings ni booleanos (salvo el '+' con algún operando string, que concatena), ni comparaciones entre tipos distintos.
- No se permiten funciones recursivas ni pipelines que, al ser aplicados, se apliquen a sí mismos.

Para el chequeo de tipos, como las columnas recién se conocen al ejecutar el programa, se agregó un tipo desconocido compatible con todos los demás, que se asigna a las columnas, a los parámetros externos y a los parámetros de las funciones. El tipo de retorno de cada función se infiere a partir de su cuerpo, mientras que los pipelines se chequean en cada APPLY con los tipos de los argumentos recibidos, de la misma forma que el cuerpo de un for se chequea una vez por cada variante.

Para la generación de código se optó por traducir el programa a Python. El código generado comienza con un conjunto fijo de funciones auxiliares (para leer y escribir los archivos, inferir el tipo de cada celda, leer los parámetros externos y verificar los require), seguido de las funciones y pipelines, las constantes y finalmente el script. Cada stage se traduce a una operación que construye una lista nueva de filas sin modificar la anterior, lo que permite implementar los bloques de ramificación simplemente partiendo de la misma lista. Para evitar conflictos con palabras reservadas de Python y entre identificadores ocultos, todo identificador se emite con el número de su símbolo como sufijo. El resultado se escribe en un archivo .py, cuyo nombre se recibe como argumento.

3.3. Adicionales (opcional)

Además de lo requerido, se agregó al compilador la opción --run, que ejecuta el programa Python generado inmediatamente después de compilarlo, pasándole los parámetros externos recibidos por línea de comandos. Esto permite pasar de un programa TabularDSL a los archivos CSV de salida en un único paso.

3.4. Dificultades Encontradas

Una de las principales dificultades encontradas fue distinguir las columnas del archivo CSV de los identificadores declarados en el programa. En un principio ambos se escribían sin prefijo, por lo que en el AST eran indistinguibles: al buscar una columna en la tabla de símbolos no se encontraba, y se reportaba como un identificador no declarado. Como las columnas recién se conocían al ejecutar el programa, no era posible resolverlo en el backend. Finalmente, se decidió darle a las columnas una sintaxis propia (@nombre), con su propio token y su propio tipo de nodo en el AST, siguiendo el mismo enfoque que ya se usaba para los parámetros externos.

Otra dificultad surgió en la gramática del for con tuplas. En la primera versión las tuplas se escribían entre paréntesis, lo que generaba una ambigüedad para un parser LALR(1), ya que una tupla y una expresión entre paréntesis comienzan de la misma forma. Esto se resolvió escribiendo las tuplas entre corchetes, de manera que el parser pudiera decidir con un único token de lookahead.

Por último, el hecho de que el esquema del CSV se desconozca al compilar impidió realizar algunas validaciones previstas en la especificación, como rechazar una columna inexistente o la aritmética sobre una columna de tipo string. Estos casos se detectan recién al ejecutar el programa generado. Por el mismo motivo, las restricciones de los require, que se buscaba validar en tiempo de compilación, se terminaron verificando al ejecutar el programa.

4. Futuras Extensiones

Una extensión que resultó particularmente interesante fue la posibilidad de leer el encabezado y el contenido del archivo CSV al compilar, cuando la ruta del LOAD es un string literal. Esto permitiría inferir el tipo de cada columna y validar al compilar tanto la existencia de las columnas como la compatibilidad de tipos en las expresiones que las utilizan, que hoy recién se detectan al ejecutar.

Otra posible extensión sería agregar nuevos tipos de datos, como fechas y horas.

También quedó pendiente incorporar el operador de negación, actualmente los literales numéricos no admiten signo, por lo que un valor negativo debe escribirse como una resta (por ejemplo, 0 - 1).

5. Conclusiones

Como conclusión del proyecto, se pudo llevar a cabo el desarrollo de un compilador, pasando por las tres etapas propuestas por la cátedra, comenzando con la idea y luego realizando tanto la parte de frontend como de backend. Las correcciones recibidas sobre la especificación inicial derivaron en varios cambios sobre el lenguaje durante el desarrollo, como la ubicación del require en la cabecera del pipeline o la sintaxis propia para las columnas. La etapa de backend fue la de mayor complejidad, en particular por el manejo de scopes y el chequeo de tipos en presencia de valores que recién se conocen al ejecutar. La generación de código, en cambio, se simplificó en gran medida al elegir Python como lenguaje de salida.

A su vez, el proyecto permitió una visualización más concreta de las temáticas estudiadas en la materia, desde la definición del dominio y el lenguaje, hasta el desarrollo de los análisis léxico, sintáctico y semántico, así como también la generación de código.

6. Bibliografía

1. (2026-08-06) Proyecto Especial
2. (2026-03-12) Modelo Computacional
3. (2025-09-03, v1.0.0) Análisis Léxico
4. (2024-05-08, v0.1.0) Análisis Sintáctico
5. (2024-09-25, v0.2.0) Análisis Semántico
6. (2024-05-16, v0.1.0) Sistema de Tipos
7. (2024-05-16, v0.1.0) Generación de Código